C PROGRAM ASSIGNMENT

Presented by: Lavanya.R
Department: computer science engineering
Collage:Jaya Shakthi engineering College

FUNDAMENTALS OF PROBLEM SOLVING AND C PROGRAMMING

TABLE OF CONTENTS

1. Fundamentals of Problem Solving

2. Introduction to C

3. Operators and Expressions

4. Input and Output Functions

INTRODUCTION TO PROBLEM SOLVING

- Problem solving is the process of finding a solution to a problem.
- It helps us understand a problem clearly.
- A computer solves problems by following instructions.
- Algorithms and flowcharts help us plan solutions.
- Problem solving is an important step in C programming.

TYPES OF PROBLEMS AND PROBLEM-SOLVING STRATEGIES

1. Mathematical Problems

2. Logical Problems

3. Real-world Problems

4. Data Processing Problems

5. Every problem needs a suitable solution strategy.

PROBLEM SOLVING WITH COMPUTERS

1. Computers solve problems by following instructions.

2. A problem must be clearly understood.

3. The solution is planned using an algorithm.

4. The algorithm is converted into a program.

5. The computer executes the program and produces the output.

STEPS IN PROBLEM SOLVING

1. Define the Problem

2. Analyze the Problem

3. Develop an Algorithm

4. Draw a Flowchart

5. Write the Program

6. Test and Debug the Program

7. Get the Final Output

INTRODUCTION TO ALGORITHMS

1. An algorithm is a step-by-step procedure to solve a problem.

2. It helps us plan a solution clearly.

3. Each step must be simple and easy to understand.

4. Algorithms are written before coding.

5. They help programmers create correct programs.

CHARACTERISTICS OF AN ALGORITHM

1. Clear and Simple Steps

2. Well-Defined Input

3. Well-Defined Output

4. Finite Number of Steps

5. Effective and Easy to Understand

6. Gives the Correct Solution

INTRODUCTION TO FLOWCHARTS

1. A flowchart is a graphical representation of an algorithm.

2. It uses symbols to explain the steps of a process.

3. Arrows show the direction of flow.

4. Flowcharts make problem-solving easier.

5. They help programmers understand program logic.

TYPES OF FLOWCHART SYMBOLS

1. Oval – Represents Start and End.

2. Rectangle – Represents a Process.

3. Parallelogram – Represents Input and Output.

4. Diamond – Represents a Decision.

5. Arrow – Shows the Direction of Flow.

PSEUDOCODE

1. Pseudocode describes program logic in simple English.

2. It does not follow strict programming language rules.

3. It helps programmers plan their programs.

4. It is easy to read and understand.

5. It can be converted into a programming language.

INTRODUCTION TO C PROGRAMMING

1. C is a general-purpose programming language.

2. It was developed by Dennis Ritchie in 1972.

3. C is widely used for system programming.

4. It supports structured programming.

5. C programs are efficient, portable, and fast.

FEATURES OF C LANGUAGE

1. Simple and Efficient

2. Structured Programming

3. Portable Language

4. Rich Set of Operators

5. Fast Execution

6. Supports Functions and Pointers

STRUCTURE OF A C- PROGRAM

1. Documentation Section

2. Header Files

3. main() Function

4. Variable Declarations

5. Executable Statements

6. Return Statement

SIMPLE C PROGRAM

```
#include <stdio.h>


int main()

{

printf("Hello World");

return 0;

}
```

Output: Hello World

C OPERATORS

1. Operators are symbols used to perform operations.

2. Arithmetic Operators: +, -, *, /, %

3. Relational Operators: ==, !=, >, <

4. Logical Operators: &&, ||, !

5. Assignment Operator: =

6. Operators help perform calculations and comparisons.

DATA TYPES IN C

1. Data types define the type of data stored in a variable.

2. int – Stores integers.

3. float – Stores decimal numbers.

4. char – Stores a single character.

5. double – Stores decimal values with greater precision.

VARIABLES AND CONSTANTS

1. A variable stores a value that can change.

2. A constant stores a fixed value.

3. Variables must be declared before use.

4. Example: int age = 18;

5. Example: const float PI = 3.14;

INPUT AND OUTPUT FUNCTIONS

1. Input functions receive data from the user.

2. scanf() is commonly used to read input.

3. Output functions display information.

4. printf() is commonly used to display output.

5. These functions are declared in the stdio.h header file.

DECISION-MAKING STATEMENTS IN C

1. Decision statements control program execution.

2. if – Executes code when a condition is true.

3. if-else – Selects between two alternatives.

4. else-if – Checks multiple conditions.

5. switch – Selects from multiple cases.

TYPES OF OPERATORS IN C

1. Arithmetic Operators – +, -, *, /, %

2. Relational Operators – ==, !=, >, <

3. Logical Operators – &&, ||, !

4. Assignment Operators – =, +=, -=

5. Increment and Decrement – ++, --

6. Operators perform calculations and comparisons.

OPERATOR PRECEDENCE IN C

1. Operator precedence determines the order of evaluation.

2. Parentheses are evaluated first.

3. Multiplication and division come before addition and subtraction.

4. Operators with the same precedence may follow associativity rules.

5. Example: 2 + 3 * 4 = 14

FORMATTED INPUT AND OUTPUT IN C

1. printf() displays formatted output.

2. scanf() reads formatted input.

3. %d represents an integer.

4. %f represents a floating-point value.

5. %c represents a character.

6. Format specifiers help read and display data correctly.

DECISION-MAKING USING IF-ELSE

1. The if statement checks a condition.

2. If the condition is true, its block executes.

3. The else block executes when the condition is false.

4. Example: if (mark >= 40)

5. It can be used to determine pass or fail.

LOOPING STATEMENTS IN C

1. Loops repeat a set of statements.

2. The for loop is useful for a known number of repetitions.

3. The while loop checks its condition before execution.

4. The do-while loop executes at least once.

5. Loops reduce code duplication and save time.

FUNCTIONS AND CODE REUSABILITY

1. A function performs a specific task.

2. Functions divide a program into smaller parts.

3. They improve readability and organization.

4. Functions help reuse code.

5. Parameters pass information to functions.

6. Return values provide results to the calling code.

ERRORS IN C PROGRAMMING

1. Syntax Errors – Violations of language rules.

2. Runtime Errors – Errors that occur during execution.

3. Logical Errors – The program runs but produces an incorrect result.

4. Testing helps identify errors.

5. Debugging helps find and correct errors.

TESTING AND DEBUGGING

1. Testing checks whether a program works correctly.

2. Different input values should be tested.

3. Expected output is compared with actual output.

4. Debugging identifies and fixes errors.

5. Testing improves program reliability.

APPLICATIONS OF C PROGRAMMING

1. Operating Systems

2. Embedded Systems

3. Device Drivers

4. Compilers and System Utilities

5. Microcontrollers and Hardware Applications

6. C is useful where efficiency and performance are important.

CONCLUSION

1. Problem-solving develops logical thinking.

2. Algorithms and flowcharts help plan solutions.

3. C programming converts solutions into executable programs.

4. Operators and input/output functions are essential in C.

5. Testing and debugging improve program quality.

Thank You!